

Number systems

We have glanced at the decimal and binary systems yesterday. There are two more useful representations that use base 8 and base 16 and are called **octal** and **hexadecimal** number systems respectively.

Important note: Base is also called *radix*.

Example: Octal System

Consider the octal number:

$$(753)_8$$

Its positional decomposition is:

$$\begin{aligned}(753)_8 &= 7 \times 8^2 + 5 \times 8^1 + 3 \times 8^0 \\ &= 7 \times 64 + 5 \times 8 + 3 \times 1 \\ &= 448 + 40 + 3 \\ &= 491_{10}\end{aligned}$$

Therefore,

$$(753)_8 = (491)_{10}$$

Example: Hexadecimal System

Consider the hexadecimal number:

$$(3A5)_{16}$$

Here, the hexadecimal digit *A* represents the decimal value 10.

$$\begin{aligned}(3A5)_{16} &= 3 \times 16^2 + 10 \times 16^1 + 5 \times 16^0 \\ &= 3 \times 256 + 10 \times 16 + 5 \times 1 \\ &= 768 + 160 + 5 \\ &= 933_{10}\end{aligned}$$

Therefore,

$$(3A5)_{16} = (933)_{10}$$

Binary number system with fractional parts

Similar to the usual decimal system we can also represent fractional parts in binary system. The representation now becomes

$$B = b_{n-1}b_{n-2} \cdots b_1b_0.b_{-1}b_{-2} \cdots b_{-m}$$

this represents

$$D = \sum_{i=-m}^{n-1} b_i 2^i$$

Examples

$$101011_2 \longleftrightarrow 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 43$$

$$(101011)_2 = (43)_{10}$$

$$.0101_2 \longleftrightarrow 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = .3125$$

$$(.0101)_2 = (.3125)_{10}$$

$$101.11_2 \longleftrightarrow 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 5.75$$

$$(101.11)_2 = (5.75)_{10}$$

Can you now convert binary numbers to decimal numbers?

Decimal to binary conversion

Consider the integer and fractional parts separately.

- Take the integer part and repeatedly divide it by 2 until you get a zero.
 - Keep on dividing the quotient by 2.
 - Write down the obtained remainders in a sequence.
 - Stop when your quotient becomes a zero.
 - Write down the sequence of remainders in the **reverse order**.
- Take the fractional part and repeatedly multiply by 2.
 - Keep on multiplying the fractional parts by 2.
 - Writing the integer parts that you get in a sequence.
 - Discard the integer part and repeat the process on fractional part.
- Write integer part then a dot and then the fractional part.

Examples

Let us represent 200 in binary.

<i>quotient</i>	<i>dividend</i>	<i>remainder</i>	
2	200	0	
2	100	0	
2	50	0	
2	25	1	
2	12	0	
2	6	0	
2	3	1	
2	1	1	
2	0	<i>Stop</i>	

$200_{10} = 11001000_2$

Notice the reverse order

Let us consider .634 to see how to handle fractional parts.

$$\begin{aligned} .634 \times 2 &= 1.268 \\ .268 \times 2 &= 0.536 \end{aligned}$$

$$\begin{aligned}
 .536 \times 2 &= 1.072 \\
 .072 \times 2 &= 0.144 \\
 .144 \times 2 &= 0.288
 \end{aligned}$$

$$(.634)_{10} = (.10100...)_{2}$$

To put both of these together, let us convert 12.25 to binary

$$\begin{aligned}
 (12)_{10} &= (1100)_2 \\
 (.25)_{10} &= (.01)_2 \\
 (12.25)_{10} &= (1100.01)_2
 \end{aligned}$$

Hexadecimal number system

This is a compact way of representing binary numbers. We assign a symbol to four bits put together as below:

0000	$\longleftrightarrow$	0
0001	$\longleftrightarrow$	1
0010	$\longleftrightarrow$	2
0011	$\longleftrightarrow$	3
0100	$\longleftrightarrow$	4
0101	$\longleftrightarrow$	5
0110	$\longleftrightarrow$	6
0111	$\longleftrightarrow$	7
1000	$\longleftrightarrow$	8
1001	$\longleftrightarrow$	9
1010	$\longleftrightarrow$	A
1011	$\longleftrightarrow$	B
1100	$\longleftrightarrow$	C
1101	$\longleftrightarrow$	D
1110	$\longleftrightarrow$	E
1111	$\longleftrightarrow$	F

Converting from binary to hexadecimal

We treat the integer and fractional parts separately as before and put them together later.

- Process the integer part from right to left, replace each consecutive group of four bits by its hexadecimal equivalent.
 - Add zeroes at the beginning if needed (called leading zeroes).
- Process the fractional part from left to right, replace each consecutive group of four bits by its hexadecimal equivalent.
 - Add zeroes at the end if needed (called trailing zeroes).

Examples

$$(1011\ 0100\ 0011)_2 = (B43)_{16}$$

Adding leading zeroes

$$(10\ 1010\ 0001)_2 = (0010\ 1010\ 0001)_2 = (2A1)_{16}$$

Adding both leading and trailing zeroes

$$(.1000\ 010)_2 = (.1000\ 0100)_2 = (.84)_{16}$$

Adding both leading and trailing zeroes

$$(101.0101\ 111)_2 = (0101.0101\ 1110)_2 = (5.5E)_{16}$$

Converting from hexadecimal to binary

Just replace each hexadecimal digit with the four bit binary equivalent.

$$\begin{aligned}(3A5)_{16} &= (0011\ 1010\ 0101)_2 \\ (12.3D)_{16} &= (0001\ 0010\ .\ 0011\ 1101)_2 \\ (1.8)_{16} &= (0001\ .\ 1000)_2\end{aligned}$$

Question

How do you convert from binary to octal and vice versa? Can you group three bits together like we did for hexadecimal numbers?

Signed and unsigned integers

Till now we seen only non-negative integers. We will also see different representations of negative integers.

Unsigned binary numbers

An n -bit binary number

$$B = b_{n-1}b_{n-2} \cdots b_2b_1b_0$$

How many possible n -bit binary numbers can we have? It is $2 \times 2 \times 2 \times \cdots \times 2$ repeated n times. Which is 2^n . We can represent numbers from 0 to $2^n - 1$.

For $n = 3$ the numbers we can represent are 000, 001, 010, 011, 100, 101, 110, 111.

As we increase the value of the number of bits n we can represent more numbers.

$$\begin{aligned}n = 8 &\rightarrow 0 \text{ to } 2^8 - 1 \text{ (255)} \\ n = 16 &\rightarrow 0 \text{ to } 2^{16} - 1 \text{ (65535)} \\ n = 32 &\rightarrow 0 \text{ to } 2^{32} - 1 \text{ (4294967295)}\end{aligned}$$

Signed binary numbers

When we also need to represent negative integers then there are three main ways.

- Sign-magnitude representation.
- One's complement representation.
- Two's complement representation.

Sign magnitude representation

Use one bit to represent the sign. This bit is generally the most significant bit (MSB). Use the rest of the bits to represent the magnitude of the number.

positive $\rightarrow 0$

negative $\rightarrow 1$

For an n -bit number

Use b_{n-1} for the sign
Use $b_{n-2} \cdots b_1 b_0$ for the magnitude

One problem with this is that there is a negative and a positive zero.

$+0 \rightarrow 0\ 00 \cdots 00$

$-0 \rightarrow 1\ 00 \cdots 00$

What is the range of numbers that can be represented in this notation? It is -2^{n-1} to 2^{n-1} .

One's complement representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in 1's complement form.

How to compute the 1's complement of a number?

- Flip every bit of the number (1 to 0 and 0 to 1). The actual term is taking the complement.
- The MSB will be the sign of the number. For positive numbers 0 and for negative numbers 1.

Positive Number	3-bit One's Complement	Negative Number	3-bit One's Complement
+0	000	-0	111
+1	001	-1	110
+2	010	-2	101
+3	011	-3	100

Notice that the range of the numbers is the same as that for sign-magnitude representation. We also have the same problem of having two different representations for zero.

Two's complement representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in two's complement form.

How to compute the two's complement of a number?

- Complement every bit of the number (1 to 0 and 0 to 1), and then add one to the resulting number.
- MSB will indicate the sign of the number.

Representation of 3-bit numbers in two's complement.

Positive Number	3-bit Two's Complement	Negative Number	3-bit Two's Complement
+0	000	-1	111
+1	001	-2	110
+2	010	-3	101
+3	011	-4	100

For example, to get the representation of -3.

- First write $+3 = 011$.
- Then Complement $011 \rightarrow 100$.
- Now add a 1 to get $100 + 1 = 101$.

Advantages

- Range is from -2^{n-1} to $2^{n-1} - 1$
- Only one representation of zero.
- Subtraction and addition can be done using the same operations.

Note: Most modern computers use this notation.

Binary arithmetic (addition and subtraction)

Before visiting the general addition and subtraction of n -bit numbers in one's complement and two's complement representations, let us first look at addition of 1-bit numbers.

Addition of single bits or binary digits follows simple rules:

- $0 + 0$ is 0 with no carry
- $0 + 1$ is 1 with no carry
- $1 + 0$ is 1 with no carry
- $1 + 1$ is 0 with a carry of 1 which is 10 if we allow two bits.

If we want to put this in a truth table form (which can help in digital logic design) we get

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Both one's complement and two's complement representations allow for using the same sequence of operations to handle both addition and subtraction.

The trick: To subtract a number b from another number a , just take $-b$ in the representation and add it to a .

Addition and subtraction for numbers in one's complement representation

We do the addition in a way very similar to decimal numbers with carry forward. The only difference is that we follow the sum and carry calculation procedure that we have just seen. Let us look closely at subtraction

- To compute $a - b$,
- Take the one's complement of b and get $-b$.
- Now add a and $-b$ using the binary addition procedure to get the result c .
- If we get a carry of 1 at the end of the procedure then we need to perform extra operations
 - Add the carry back to c . This is called *end-around carry*.
 - This means that final result $r = c + 1$.
 - The result will be a positive number.
- If the carry at the end of the procedure is 0 then no extra processing is needed. The result will be a negative number in its one's complement representation.

Example with end-around carry:

```

      Carry:  1 1 0  (carry into each column)
              ↓ ↓ ↓
              0 1 1  (+3)
            + 1 0 1  (-2)
            -----
              1 0 0 0
  
```

Now perform end-around carry

```

              0 0 0
            + 0 0 1
            -----
      Result:  0 0 1  (+1)
  
```

Example without end-around carry:

```

      0011 (+3)
      1010 (-5)
      -----
      1101 (+2)
  
```

Addition and subtraction for numbers in two's complement representation

Addition is same as before. Let us look at subtraction

- To compute $a - b$,
- Get the two's complement of b as $-b$.
- Now perform addition on a and $-b$.
- If the carry at the end of the procedure is 1:
 - Ignore the carry.
 - The result is a positive number.

- If the carry at the end of the procedure is 0:
 - The result is a negative number in its two's complement form.

Example with carry:

```

    0110 (+6)
    1110 (-2)
    -----
    1 0100 (+4)
    Just ignore the carry
  
```

Example without carry:

```

    0011 (+3)
    1011 (-5)
    -----
    1110 (-2)
  
```

Underflow and overflow

Due to the limited or finite number of bits we use to represent numbers we might face situations of *underflow* and *overflow*. They are defined as:

- Adding two positive numbers should not give a negative number. If it does then it is termed as *overflow*.
- Adding two negative numbers should not give a positive number. If it does then it is termed as *underflow*.

To detect underflow or overflow check if:

- carry in and carry out of the MSB are different.

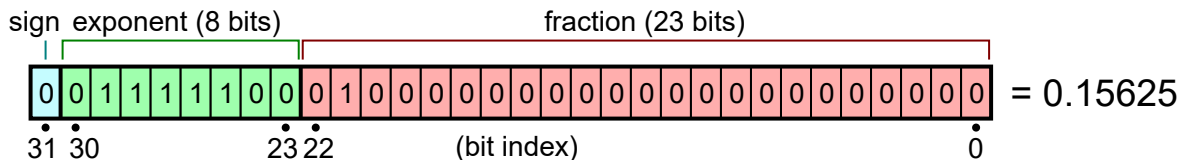
Overflow example:

```

(64)  01000000
(96)  01100000
-----
(-96) 10100000
  
```

Floating point representation

To represent a wide range of real numbers we use the floating point representation. This has been standardized by IEEE (Institute of Electrical and Electronics Engineers) under the *IEEE 754 standard*.



- Normalized numbers: $(-1)^s \times 1.f \times 2^{E-127}$
- Denormalized numbers: Here $E = 0$ and the number is $(-1)^s \times 0.f \times 2^{-126}$.
- We get zero (both negative and positive versions) when both E and f are zeroes.
- Special numbers:
 - 0 11111111 000000000000000000000000 <--> +Inf
 - 1 11111111 000000000000000000000000 <--> -Inf
 - 0 11111111 Any non-zero value <--> NaN
 - 1 11111111 Any non-zero value <--> NaN
 - Inf is infinity.
 - NaN is Not a number.

For more precision and range we can increase the size of the representation to 64-bit and this is usually called a *double*.

